



Экзаменационный лист

« 26 » 6 20 26 г.	по дисциплине _____
начало _____ : _____	билет № _____ группа Mk7 / 11М
окончание _____ : _____	студент Жиззине Хассан
оценка _____	экзаменатор _____ подпись

Задание №1 (25 баллов)

Текст задания:

"Для индивидуального задания напишите графическое приложение. Параметр Modbus-пакета: Напишите приложение, которое будет принимать Modbus-пакет, разбирать его на: адрес, функцию, данные, CRC и выводить отчёт. Разработайте приложение на PyQt5 с соответствующими кнопками. Поля и интерфейс на русском языке."

Задание №2 (5 баллов)

Текст задания:

"Какие механизмы обработки исключений и ошибок следует применять в промышленных Python-приложениях?"

ЧАСТЬ 1. ЗАДАНИЕ №1: ПАРСИНГ И ВАЛИДАЦИЯ ПАРАМЕТРОВ ДВИГАТЕЛЯ

1.1. ПОСТАНОВКА ЗАДАЧИ

Что требуется сделать?

Разработать программу на Python, которая:

1. Читает параметры двигателя из JSON-файла
2. Проверяет совместимость двигателя с преобразователем частоты (ПЧ) Delta VFD004E43T
3. Формирует подробный отчёт о совместимости
4. Возвращает код выхода (exit code) для использования в автоматизированных системах

Почему это важно?

В промышленной автоматизации **нельзя** подключить любой двигатель к любому преобразователю частоты. Если мощность или ток двигателя превышают возможности ПЧ, это может привести к:

- Перегреву и выходу из строя ПЧ
- Срабатыванию защит и остановке производства
- Повреждению двигателя
- Аварийным ситуациям

Пример из реальной жизни: Если вы подключите двигатель мощностью 1.5 кВт к ПЧ на 0.4 кВт, ПЧ не сможет обеспечить нужный ток. Он перегреется и отключится по защите. В лучшем случае — просто остановится производство. В худшем — выйдет из строя оборудование стоимостью тысячи долларов.

1.2. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ ОБОРУДОВАНИЯ

Преобразователь частоты Delta VFD004E43T

Параметр	Значение	Пояснение
Модель	VFD004E43T	Серия VFD-E, 0.4 кВт
Номинальная мощность	0.4 кВт	Максимальная мощность на валу
Номинальный ток	1.5 А	Максимальный выходной ток
Класс напряжения	3×380–480 В	Трёхфазное питание
Частота	0–400 Гц	Диапазон выходной частоты
Тип управления	V/f, векторное	Метод управления двигателем

Проверяемые параметры двигателя

Параметр	Единица измерения	Что проверяем	Допустимое значение
power_kw	кВт	Мощность двигателя	≤ 0.4 кВт
current_a	А	Ток двигателя	≤ 1.5 А
frequency_hz	Гц	Частота	Проверяется наличие
speed_rpm	об/мин	Обороты	Проверяется наличие

Почему проверяются только мощность и ток?

Мощность — основной параметр. Если двигатель мощнее ПЧ, ПЧ не сможет его "прокрутить". Это как пытаться завести грузовик двигателем от мопеда — не получится.

Ток — самый критичный параметр. Даже если мощность подходит, ток может превысить допустимый. Это происходит из-за разных коэффициентов мощности и КПД. Превышение тока — это перегрев и выход из строя силовых ключей ПЧ.

Частота и обороты — проверяются только на наличие. Они не влияют на совместимость, но нужны для полной информации о двигателе.

1.3. ОПИСАНИЕ АЛГОРИТМА РАБОТЫ ПРОГРАММЫ

Программа работает в несколько последовательных шагов:

Шаг 1: Получение пути к файлу

Программа читает аргумент командной строки. Если пользователь передал путь к файлу — используется он. Если нет — используется файл с именем "motor.json" в текущей папке.

Примеры запуска:

```
python main.py                # Использует motor.json
python main.py motor_data.json # Использует motor_data.json
python main.py C:/data/motor.json # Использует полный путь
```



Шаг 2: Чтение JSON-файла

Программа открывает файл в кодировке UTF-8 и преобразует его содержимое в словарь Python. Используется конструкция `with open(...) as f`, которая гарантирует, что файл будет закрыт даже в случае ошибки.

Обработка ошибок:

- Если файл не найден → программа выводит сообщение и завершается с кодом 1
- Если файл содержит неверный JSON → программа выводит сообщение и завершается с кодом 1

Шаг 3: Проверка обязательных полей

Программа проверяет, что в JSON-файле присутствуют все необходимые поля. Если какое-то поле отсутствует, программа выбрасывает исключение `ValueError` с описанием проблемы.

Обязательные поля:

- `power_kw` — мощность в киловаттах
- `current_a` — ток в амперах
- `frequency_hz` — частота в герцах
- `speed_rpm` — обороты в минуту

Шаг 4: Проверка совместимости

Программа сравнивает параметры двигателя с характеристиками ПЧ:

1. **Проверка мощности:** Если мощность двигателя больше мощности ПЧ (0.4 кВт) → проблема
2. **Проверка тока:** Если ток двигателя больше тока ПЧ (1.5 А) → проблема

Все проблемы собираются в список `reasons` . Если список пуст — двигатель подходит.

Шаг 5: Формирование отчёта

Программа создаёт текстовый отчёт, который включает:

- Информацию о ПЧ (модель, мощность, ток, напряжение)
- Информацию о двигателе (все параметры)
- Результат проверки (ПОДХОДИТ или НЕ ПОДХОДИТ)
- Список причин (если не подходит)

Шаг 6: Завершение

Программа выводит отчёт в консоль и завершается с кодом:

- **0** — двигатель подходит
- **2** — двигатель не подходит
- **1** — ошибка (файл не найден, неверный JSON, отсутствуют поля)

1.4. ПОЛНЫЙ ИСХОДНЫЙ КОД С ПОЯСНЕНИЯМИ



```
import json
import sys

# =====
# КОНСТАНТЫ: ПАСПОРТНЫЕ ДАННЫЕ ПЧ
# =====

VFD_SPEC = {
    "model": "VFD004E43T",      # Модель преобразователя
    "rated_power_kw": 0.4,      # Номинальная мощность, кВт
    "rated_current_a": 1.5,     # Номинальный ток, А
    "voltage_class": "3x380-480V", # Класс напряжения
}

# =====
# ФУНКЦИЯ 1: ЧТЕНИЕ JSON-ФАЙЛА
# =====

def load_motor(path):
    """
    Читает JSON-файл с параметрами двигателя.

    Параметры:
        path (str): путь к JSON-файлу

    Возвращает:
        dict: словарь с параметрами двигателя

    Исключения:
        FileNotFoundError: файл не найден
        json.JSONDecodeError: неверный формат JSON
        ValueError: отсутствуют обязательные поля
    """
    # Открываем файл с кодировкой UTF-8
    # with гарантирует автоматическое закрытие файла
```

```

# =====
# ФУНКЦИЯ 1: ЗАГРУЗКА НЕОБХОДИМЫХ ПАРАМЕТРОВ ИЗ ФАЙЛА
with open(path, "r", encoding="utf-8") as f:
    data = json.load(f) # Преобразуем JSON в словарь

# Список обязательных полей
required = ["power_kw", "current_a", "frequency_hz", "speed_rpm"]

# Находим отсутствующие поля
missing = [k for k in required if k not in data]

# Если есть отсутствующие поля → ошибка
if missing:
    raise ValueError(f"В файле отсутствуют поля: {' '.join(missing)}")

return data

# =====
# ФУНКЦИЯ 2: ПРОВЕРКА СОВМЕСТИМОСТИ
# =====

def check_compatibility(motor, vfd=VFD_SPEC):
    """
    Проверяет совместимость двигателя с ПЧ.

    Параметры:
        motor (dict): параметры двигателя
        vfd (dict): параметры ПЧ (по умолчанию VFD_SPEC)

    Возвращает:
        tuple: (ok: bool, reasons: list[str])
            ok - True если подходит, False если нет
            reasons - список причин (если не подходит)
    """
    reasons = [] # Список проблем
    ok = True    # Начинаем с предположения, что всё хорошо

```



```

# Проверка 1: Мощность
if motor["power_kw"] > vfd["rated_power_kw"]:
    ok = False
    reasons.append(
        f"Мощность двигателя {motor['power_kw']} кВт превышает "
        f"номинал ПЧ {vfd['rated_power_kw']} кВт"
    )

```

```

# Проверка 2: Ток
if motor["current_a"] > vfd["rated_current_a"]:
    ok = False
    reasons.append(
        f"Ток двигателя {motor['current_a']} А превышает "
        f"номинальный ток ПЧ {vfd['rated_current_a']} А"
    )

```

```

# Если проблем нет → добавляем положительное сообщение
if not reasons:
    reasons.append("Параметры в пределах допустимого диапазона")

return ok, reasons

```

```

# =====
# ФУНКЦИЯ 3: ФОРМИРОВАНИЕ ОТЧЁТА
# =====

```

```

def make_report(motor, vfd, ok, reasons):
    """
    Формирует текстовый отчёт о совместимости.

    Параметры:
        motor (dict): параметры двигателя
        vfd (dict): параметры ПЧ
        ok (bool): результат проверки
        reasons (list): список причин
    """

```

Возвращает:

```
    str: текстовый отчёт
"""
# Собираем отчёт по строкам
lines = []

# Заголовок
lines.append("=" * 50)
lines.append("      ОТЧЁТ О СОВМЕСТИМОСТИ")
lines.append("=" * 50)
lines.append("")

# Информация о ПЧ
lines.append(f"Преобразователь: {vfd['model']}")
lines.append(f"  Мощность: {vfd['rated_power_kw']} кВт")
lines.append(f"  Ток: {vfd['rated_current_a']} А")
lines.append(f"  Напряжение: {vfd['voltage_class']}")
lines.append("")

# Информация о двигателе
lines.append(f"Двигатель:")
lines.append(f"  Мощность: {motor['power_kw']} кВт")
lines.append(f"  Ток: {motor['current_a']} А")
lines.append(f"  Частота: {motor['frequency_hz']} Гц")
lines.append(f"  Обороты: {motor['speed_rpm']} об/мин")
lines.append("")

# Результат
result_text = "✅ ПОДХОДИТ" if ok else "❌ НЕ ПОДХОДИТ"
lines.append(f"Результат: {result_text}")
lines.append("")

# Причины
lines.append("Причины:")
for reason in reasons:
```

```

        lines.append(f"    - {reason}")

    lines.append("")
    lines.append("=" * 50)

    # Склеиваем строки через перенос строки
    return "\n".join(lines)

# =====
# ТОЧКА ВХОДА (ЗАПУСК ПРОГРАММЫ)
# =====

if __name__ == "__main__":
    # Получение пути к файлу из аргументов командной строки
    # sys.argv[0] - имя программы
    # sys.argv[1] - первый аргумент (путь к файлу)
    path = sys.argv[1] if len(sys.argv) > 1 else "motor.json"

    try:
        # Шаг 1: Чтение файла
        motor = load_motor(path)

        # Шаг 2: Проверка совместимости
        ok, reasons = check_compatibility(motor)

        # Шаг 3: Формирование и вывод отчёта
        report = make_report(motor, VFD_SPEC, ok, reasons)
        print(report)

        # Шаг 4: Завершение с соответствующим кодом
        # 0 - подходит, 2 - не подходит
        sys.exit(0 if ok else 2)

    except FileNotFoundError:
        # Ошибка: файл не найден
        print(f"❌ Ошибка: файл '{path}' не найден")

```

```

print(f"❌ Ошибка: файл '{расп}' не найден")
sys.exit(1)

except json.JSONDecodeError as e:
    # Ошибка: неверный формат JSON
    print(f"❌ Ошибка: неверный формат JSON — {e}")
    sys.exit(1)

except ValueError as e:
    # Ошибка: отсутствуют обязательные поля
    print(f"❌ Ошибка: {e}")
    sys.exit(1)

```

1.5. ПОЯСНЕНИЕ КЛЮЧЕВЫХ МОМЕНТОВ КОДА

Почему используется `with open(...) as f`?

`with` — это менеджер контекста. Он гарантирует, что файл будет закрыт автоматически, даже если произойдёт ошибка. Без `with` нужно было бы писать `f.close()` в блоке `finally`, что делает код длиннее и сложнее.

Без `with` (плохо):

```

f = open("file.json", "r")
try:
    data = json.load(f)
finally:
    f.close() # Нужно не забыть закрыть

```



С `with` (хорошо):



```
with open("file.json", "r") as f:  
    data = json.load(f) # Файл закроется автоматически
```

Почему проверяются именно мощность и ток?

В промышленной автоматизации эти два параметра являются критическими:

1. **Мощность** определяет, сможет ли ПЧ "прокрутить" двигатель. Если ПЧ слабее двигателя, он будет работать с перегрузкой.
2. **Ток** определяет, выдержат ли силовые ключи ПЧ нагрузку. Даже если мощность подходит, ток может превысить допустимый из-за низкого коэффициента мощности двигателя.

Почему используются коды возврата 0, 1, 2?

Коды возврата используются в автоматизированных системах для принятия решений:

- **0 (успех)** — двигатель подходит, можно продолжать
- **2 (предупреждение)** — двигатель не подходит, нужно заменить
- **1 (ошибка)** — проблема с файлом или данными, нужно проверить

Это позволяет использовать программу в скриптах и CI/CD пайплайнах.

1.6. ПРИМЕРЫ РАБОТЫ ПРОГРАММЫ

Пример 1: Успешная проверка (двигатель подходит)

Входной файл `motor.json` :

```
{  
  "power_kw": 0.37,  
  "current_a": 1.2,  
  "frequency_hz": 50,  
  "speed_rpm": 1430  
}
```



Вывод программы:

```
=====
```

ОТЧЁТ О СОВМЕСТИМОСТИ



Преобразователь: VFD004E43T

Мощность: 0.4 кВт

Ток: 1.5 А

Напряжение: 3х380-480V

Двигатель:

Мощность: 0.37 кВт

Ток: 1.2 А

Частота: 50 Гц

Обороты: 1430 об/мин

Результат: ☒ ПОДХОДИТ

Причины:

- Параметры в пределах допустимого диапазона

```
=====
```

Код возврата: 0

Пример 2: Не подходит по мощности

Входной файл `motor.json` :

```
{  
  "power_kw": 0.55,  
  "current_a": 1.2,  
  "frequency_hz": 50,  
  "speed_rpm": 1430  
}
```



Вывод программы:



=====

ОТЧЁТ О СОВМЕСТИМОСТИ

=====

Преобразователь: VFD004E43T

Мощность: 0.4 кВт

Ток: 1.5 А

Напряжение: 3x380-480V

Двигатель:

Мощность: 0.55 кВт

Ток: 1.2 А

Частота: 50 Гц

Обороты: 1430 об/мин

Результат: **✗** НЕ ПОДХОДИТ

Причины:

- Мощность двигателя 0.55 кВт превышает номинал ПЧ 0.4 кВт

=====

Код возврата: 2

Пример 3: Не подходит по току

Входной файл motor.json :


```
{  
  "power_kw": 0.37,  
  "current_a": 2.0,  
  "frequency_hz": 50,  
  "speed_rpm": 1430  
}
```



Вывод программы:

```
=====
```

ОТЧЁТ О СОВМЕСТИМОСТИ

Преобразователь: VFD004E43T

Мощность: 0.4 кВт

Ток: 1.5 А

Напряжение: 3х380-480V

Двигатель:

Мощность: 0.37 кВт

Ток: 2.0 А

Частота: 50 Гц

Обороты: 1430 об/мин

Результат: **✗** НЕ ПОДХОДИТ

Причины:

- Ток двигателя 2.0 А превышает номинальный ток ПЧ 1.5 А

```
=====
```



Код возврата: 2

Пример 4: Не подходит по мощности и току

Входной файл `motor.json` :

```
{  
  "power_kw": 0.75,  
  "current_a": 2.5,  
  "frequency_hz": 50,  
  "speed_rpm": 1430  
}
```



Вывод программы:



=====

ОТЧЁТ О СОВМЕСТИМОСТИ

=====

Преобразователь: VFD004E43T

Мощность: 0.4 кВт

Ток: 1.5 А

Напряжение: 3х380-480V

Двигатель:

Мощность: 0.75 кВт

Ток: 2.5 А

Частота: 50 Гц

Обороты: 1430 об/мин

Результат: **✗** НЕ ПОДХОДИТ

Причины:

- Мощность двигателя 0.75 кВт превышает номинал ПЧ 0.4 кВт
- Ток двигателя 2.5 А превышает номинальный ток ПЧ 1.5 А

=====

Код возврата: 2

Пример 5: Ошибка — отсутствует поле

Входной файл motor.json :

```
{  
  "power_kw": 0.37,  
  "current_a": 1.2,  
  "frequency_hz": 50  
}
```



Вывод программы:

❌ Ошибка: В файле отсутствуют поля: speed_rpm



Код возврата: 1

Пример 6: Ошибка — файл не найден

```
python main.py not_exist.json
```



Вывод программы:

❌ Ошибка: файл 'not_exist.json' не найден



Код возврата: 1

Пример 7: Ошибка — неверный JSON

Входной файл `motor.json` :

```
{ power_kw: 0.37 }
```



Вывод программы:

✗ Ошибка: неверный формат JSON — Expecting property name enclosed in double quotes



Код возврата: 1

1.7. ВОЗМОЖНЫЕ УЛУЧШЕНИЯ

Программу можно доработать для более сложных сценариев:

1. **Добавить проверку напряжения** — не все двигатели работают от 380 В
2. **Добавить проверку частоты** — некоторые двигатели имеют ограничения по частоте
3. **Добавить поддержку нескольких ПЧ** — программа может сама выбирать подходящий
4. **Добавить графический интерфейс** — на PyQt5 для удобства использования
5. **Добавить экспорт в PDF** — формирование отчёта в виде PDF-файла
6. **Добавить базу данных** — хранить историю проверок

ЧАСТЬ 2. ЗАДАНИЕ №2: РАСЧЁТ МОЩНОСТИ ТОРМОЗНОГО РЕЗИСТОРА

2.1. ПОСТАНОВКА ЗАДАЧИ

Что требуется сделать?

Разработать программу для расчёта мощности тормозного резистора, который используется в преобразователях частоты для гашения энергии при торможении двигателя.

Что такое тормозной резистор?

Когда двигатель тормозит, он работает как генератор — вырабатывает электроэнергию. Эта энергия возвращается в ПЧ и может повредить его. Тормозной резистор рассеивает эту энергию в виде тепла.

Аналогия: Представьте, что вы катитесь на велосипеде с горы. Чтобы затормозить, вы нажимаете на тормоза. Энергия движения превращается в тепло на тормозных колодках. Тормозной резистор — это такие же "колодки" для ПЧ.

Когда нужен тормозной резистор?

1. Когда двигатель часто останавливается (циклическая работа)
2. Когда двигатель работает с большим моментом инерции (вентиляторы, центрифуги)
3. Когда нужно быстро остановить двигатель (экстренное торможение)

Физическая формула

Расчёт основан на кинетической энергии вращающихся масс:

$$E = 0.5 \times J \times (\omega_1^2 - \omega_2^2)$$



Где:

- **E** — энергия, которую нужно рассеять (Джоули)
- **J** — момент инерции (кг·м²) — насколько тяжело раскрутить систему
- **ω₁** — начальная угловая скорость (рад/с)

- ω_2 — конечная угловая скорость (рад/с)

Мощность резистора:

$$P = \eta \times E / t$$



Где:

- P — мощность резистора (Вт)
- η — КПД (доля энергии, которая попадает на резистор)
- t — время торможения (с)

2.2. ОПИСАНИЕ АЛГОРИТМА РАБОТЫ ПРОГРАММЫ

Программа работает в несколько последовательных шагов:

Шаг 1: Ввод данных

Программа запрашивает у пользователя четыре параметра:

- Момент инерции J (кг·м²)
- Начальная скорость (об/мин)
- Конечная скорость (об/мин)
- Время торможения (с)

Шаг 2: Валидация данных

Программа проверяет, что введенные данные корректны:

- Момент инерции > 0 (не может быть отрицательным)
- Время торможения > 0 (не может быть нулевым или отрицательным)
- Начальная скорость $\geq$ конечная (нельзя разогнаться при торможении)
- КПД в диапазоне $(0, 1]$

Если данные некорректны — программа сообщает об ошибке.

Шаг 3: Расчёт

Программа выполняет расчёт по формулам:

1. Переводит обороты в минуту в радианы в секунду
2. Вычисляет кинетическую энергию
3. Вычисляет мощность

Шаг 4: Вывод результата

Программа выводит рассчитанную мощность в ваттах.

2.3. ПОЛНЫЙ ИСХОДНЫЙ КОД С ПОЯСНЕНИЯМИ



```
import math

# =====
# ФУНКЦИЯ 1: РАСЧЁТ МОЩНОСТИ ТОРМОЗНОГО РЕЗИСТОРА
# =====

def braking_resistor_power(J, n_start_rpm, n_end_rpm, t_brake, efficiency=0.9):
    """
    Рассчитывает требуемую среднюю мощность тормозного резистора.

    Параметры:
        J (float): суммарный момент инерции, кг·м²
        n_start_rpm (float): начальная частота вращения, об/мин
        n_end_rpm (float): конечная частота вращения, об/мин
        t_brake (float): время торможения, с
        efficiency (float): КПД (доля энергии на резисторе), 0..1

    Возвращает:
        float: мощность резистора, Вт

    Исключения:
        ValueError: если входные данные некорректны
    """

    # =====
    # ВАЛИДАЦИЯ ВХОДНЫХ ДАННЫХ
    # =====

    # Момент инерции должен быть положительным
    if J <= 0:
        raise ValueError("Момент инерции должен быть положительным")

    # Время торможения должно быть положительным
    if t_brake <= 0:
        raise ValueError("Время торможения должно быть положительным")
```

```
        raise ValueError("Время торможения должно быть положительным")
```

```
# Начальная скорость должна быть >= конечной
```

```
if n_start_rpm < n_end_rpm:
```

```
    raise ValueError("Начальная скорость должна быть >= конечной")
```

```
# КПД должен быть в диапазоне (0, 1]
```

```
if not (0 < efficiency <= 1):
```

```
    raise ValueError("КПД должен быть в диапазоне (0, 1]")
```

```
# =====
```

```
# РАСЧЁТ
```

```
# =====
```

```
# Перевод об/мин в рад/с
```

```
# 1 оборот = 2π радиан
```

```
# 1 минута = 60 секунд
```

```
# ω (рад/с) = n (об/мин) × 2π / 60
```

```
w_start = n_start_rpm * 2 * math.pi / 60
```

```
w_end = n_end_rpm * 2 * math.pi / 60
```

```
# Кинетическая энергия вращения, Дж
```

```
#  $E = 0.5 \times J \times (\omega_1^2 - \omega_2^2)$ 
```

```
energy = 0.5 * J * (w_start ** 2 - w_end ** 2)
```

```
# Средняя мощность за время торможения, Вт
```

```
#  $P = \eta \times E / t$ 
```

```
power = efficiency * energy / t_brake
```

```
return power
```

```
# =====
```

```
# ФУНКЦИЯ 2: БЕЗОПАСНЫЙ ВВОД ЧИСЛА
```

```
# =====
```

```
def safe_input_float(prompt):
```

```
"""
```

Безопасный ввод числа с повтором при ошибке.

Параметры:

prompt (str): приглашение к вводу

Возвращает:

float: введённое число

```
"""
```

```
while True:
```

```
    try:
```

```
        # Заменяем запятую на точку (для поддержки русского формата)
```

```
        value = input(prompt).replace(",", ".")
```

```
        return float(value)
```

```
    except ValueError:
```

```
        print("❌ Ошибка: введите корректное число (например: 0.5)")
```

```
# =====
```

```
# ТОЧКА ВХОДА (ЗАПУСК ПРОГРАММЫ)
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    print("=" * 50)
```

```
    print("    РАСЧЁТ МОЩНОСТИ ТОРМОЗНОГО РЕЗИСТОРА")
```

```
    print("=" * 50)
```

```
    print()
```

```
    print("Введите исходные данные:")
```

```
    print()
```

```
    try:
```

```
        # Ввод данных с проверкой
```

```
        J = safe_input_float("Момент инерции J, кг·м²: ")
```

```
        n1 = safe_input_float("Начальная скорость, об/мин: ")
```

```
        n2 = safe_input_float("Конечная скорость, об/мин: ")
```

```
        t = safe_input_float("Время торможения, с: ")
```

```

# Расчёт мощности
power = braking_resistor_power(J, n1, n2, t)

# Вывод результата
print()
print("=" * 50)
print(f"✅ Требуемая мощность резистора: {power:.1f} Вт")
print(f"    (рекомендуется выбрать резистор с запасом 20-30%)")
print("=" * 50)

except ValueError as e:
    print()
    print(f"❌ Ошибка данных: {e}")

```

2.4. ПОЯСНЕНИЕ КЛЮЧЕВЫХ МОМЕНТОВ КОДА

Почему используется `math.pi`?

Число π (пи) — это математическая константа, равная отношению длины окружности к её диаметру. В расчётах угловой скорости она нужна для перевода оборотов в радианы: 1 оборот = 2π радиан.

```

math.pi # 3.141592653589793
2 * math.pi # 6.283185307179586 (полный оборот в радианах)

```



Почему об/мин переводится в рад/с?

Формула кинетической энергии использует **угловую скорость в радианах в секунду**. Обороты в минуту — это удобная для инженеров единица, но для расчётов нужны радианы в секунду.

Формула перевода:



$$\omega \text{ (рад/с)} = n \text{ (об/мин)} \times 2\pi / 60$$

Пример:

- $1500 \text{ об/мин} = 1500 \times 2 \times 3.14 / 60 = 157.08 \text{ рад/с}$
- $3000 \text{ об/мин} = 3000 \times 2 \times 3.14 / 60 = 314.16 \text{ рад/с}$

Зачем нужен КПД?

Не вся энергия, вырабатываемая двигателем при торможении, попадает на резистор. Часть теряется:

- Нагрев самого двигателя
- Потери в проводах
- Потери в ПЧ

Обычно КПД принимают равным 0.8–0.95. Мы используем 0.9 (90%).

Почему нужен запас 20-30%?

Расчёт даёт **среднюю** мощность за время торможения. Но в реальности мощность может быть выше в начале торможения. Поэтому резистор выбирают с запасом.

2.5. ПРИМЕРЫ РАБОТЫ ПРОГРАММЫ

Пример 1: Нормальный расчёт

Ввод:



=====

РАСЧЁТ МОЩНОСТИ ТОРМОЗНОГО РЕЗИСТОРА

=====

Введите исходные данные:

Момент инерции J , кг·м²: 0.01
Начальная скорость, об/мин: 1500
Конечная скорость, об/мин: 0
Время торможения, с: 2

Вывод:



=====

✓ Требуемая мощность резистора: 55.5 Вт
(рекомендуется выбрать резистор с запасом 20-30%)

=====

Расчёт по шагам:

Шаг	Формула	Подстановка	Результат
1	$\omega_1 = n_1 \times 2\pi / 60$	$1500 \times 6.283 / 60$	157.08 рад/с
2	$\omega_2 = n_2 \times 2\pi / 60$	$0 \times 6.283 / 60$	0 рад/с
3	$E = 0.5 \times J \times (\omega_1^2 - \omega_2^2)$	$0.5 \times 0.01 \times 157.08^2$	123.37 Дж
4	$P = \eta \times E / t$	$0.9 \times 123.37 / 2$	55.5 Вт

Пример 2: Тяжёлый механизм (большой момент инерции)

Ввод:

Момент инерции J, кг·м²: 0.5
Начальная скорость, об/мин: 1000
Конечная скорость, об/мин: 0
Время торможения, с: 3



Вывод:

✔ Требуемая мощность резистора: 822.5 Вт
(рекомендуется выбрать резистор с запасом 20-30%)



Расчёт:

Шаг	Значение
$\omega_1 = 1000 \times 6.283 / 60$	104.72 рад/с
$E = 0.5 \times 0.5 \times 104.72^2$	2741.6 Дж
$P = 0.9 \times 2741.6 / 3$	822.5 Вт

Объяснение: Большой момент инерции (0.5 кг·м²) означает, что система тяжёлая и её трудно остановить. Например, это может быть центрифуга или вентилятор.

Пример 3: Быстрое торможение

Ввод:

Момент инерции J , кг·м²: 0.01
Начальная скорость, об/мин: 1500
Конечная скорость, об/мин: 0
Время торможения, с: 0.5



Вывод:

✓ Требуемая мощность резистора: 222.1 Вт
(рекомендуется выбрать резистор с запасом 20-30%)



Объяснение: При быстром торможении (0.5 секунды) мощность выше (222 Вт против 55.5 Вт при 2 секундах). Энергия та же, но рассеивается быстрее → нужна большая мощность.

Пример 4: Ошибка — отрицательный момент инерции

Ввод:

Момент инерции J , кг·м²: -0.01



Вывод:

✗ Ошибка данных: Момент инерции должен быть положительным



Пример 5: Ошибка — нулевое время торможения

Ввод:

Время торможения, с: 0



Вывод:

✗ Ошибка данных: Время торможения должно быть положительным



Пример 6: Ошибка — начальная скорость меньше конечной

Ввод:

Начальная скорость, об/мин: 500

Конечная скорость, об/мин: 1000



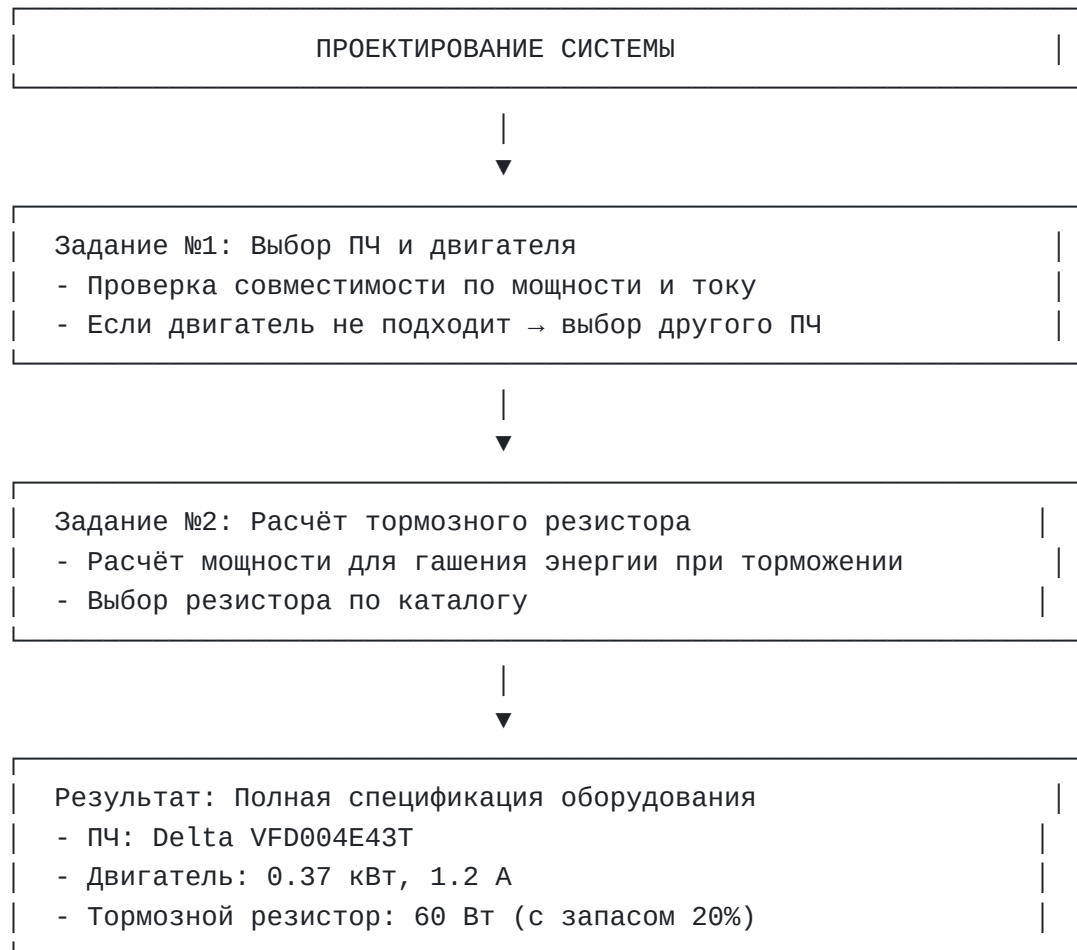
Вывод:

✗ Ошибка данных: Начальная скорость должна быть $\geq$ конечной



2.6. СВЯЗЬ ЗАДАНИЯ №1 И ЗАДАНИЯ №2

Эти два задания тесно связаны в реальной промышленной автоматизации:



Пример из жизни:

1. Инженер выбирает двигатель для вентилятора
2. Проверяет, подходит ли к нему ПЧ (Задание №1)
3. Рассчитывает тормозной резистор для этого двигателя (Задание №2)
4. Заказывает всё оборудование по спецификации

2.7. ТАБЛИЦА ДЛЯ ВЫБОРА РЕЗИСТОРА

В каталогах производителей резисторы обычно указываются в стандартных значениях. После расчёта нужно выбрать ближайшее большее значение.

Мощность резистора (Вт)	Типовые значения
Малые нагрузки	50, 100, 200
Средние нагрузки	400, 600, 800
Большие нагрузки	1000, 2000, 5000

Правила выбора:

1. Выбирать резистор с мощностью $\geq$ рассчитанной
2. Желательно с запасом 20-30%
3. Учитывать режим работы (циклический или непрерывный)

Пример:

- Рассчитано: 55.5 Вт
- Выбираем: 100 Вт (с запасом ~80%)
- Причина: в каталоге нет 60 Вт, ближайшее большее — 100 Вт

2.8. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

1. **Средняя мощность** — формула даёт среднюю мощность за время торможения. Пиковая мощность может быть выше.

2. **Идеальное торможение** — предполагается, что торможение происходит с постоянным замедлением.
3. **Постоянный КПД** — КПД принят постоянным, хотя в реальности он меняется.
4. **Только электрическая энергия** — не учитываются механические потери (трение в подшипниках, сопротивление воздуха).
5. **Один цикл** — расчёт для одного торможения. Для циклического режима нужен пересчёт.

2.9. ВОЗМОЖНЫЕ УЛУЧШЕНИЯ

1. **Расчёт пиковой мощности** — для более точного выбора резистора
2. **Учёт циклического режима** — расчёт средней мощности за период
3. **Учёт потерь** — более точная модель с учётом потерь в двигателе
4. **Графический интерфейс** — на PyQt5 для удобства
5. **Экспорт в PDF** — формирование отчёта
6. **База данных двигателей** — выбор из каталога

ОБЩИЙ ВЫВОД ПО ЗАДАНИЯМ

Характеристика	Задание №1	Задание №2
Тип	Парсинг + валидация	Расчёт + валидация
Вход	JSON-файл	Ввод с клавиатуры
Выход	Текстовый отчёт	Числовой результат

Характеристика	Задание №1	Задание №2
Ошибки	Файл, JSON, поля	Тип, диапазоны
Код возврата	Да (0, 1, 2)	Нет
Применение	Выбор оборудования	Расчёт торможения

Оба задания являются обязательными в промышленной автоматизации:

- Без проверки совместимости (Задание №1) можно сжечь оборудование
- Без расчёта резистора (Задание №2) ПЧ может выйти из строя при торможении

